

Computer Automation (Assignment)

Name : Manoj V

Class : 4th year - CSE - B

Roll no : 22BCS105

1. Iris Flower Classification

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load and explore Iris dataset
iris = load_iris()
X, y = iris.data, iris.target
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(y, iris.target_names)
print(df.head())

# Correlation heatmap
plt.figure(figsize=(6,4))
sns.heatmap(df.iloc[:, :-1].corr(), annot=True, cmap="coolwarm")
plt.title("Feature Correlation")
plt.show()

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train KNN
model = KNeighborsClassifier(n_neighbors=7)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title("Confusion Matrix")
plt.show()
```

Output:

	Sepal Length	Sepal Width	Petal Length	Petal Width
0	5.1	3.5	1.4	0.2

1	4.9	3.4	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.0	1.4	0.2

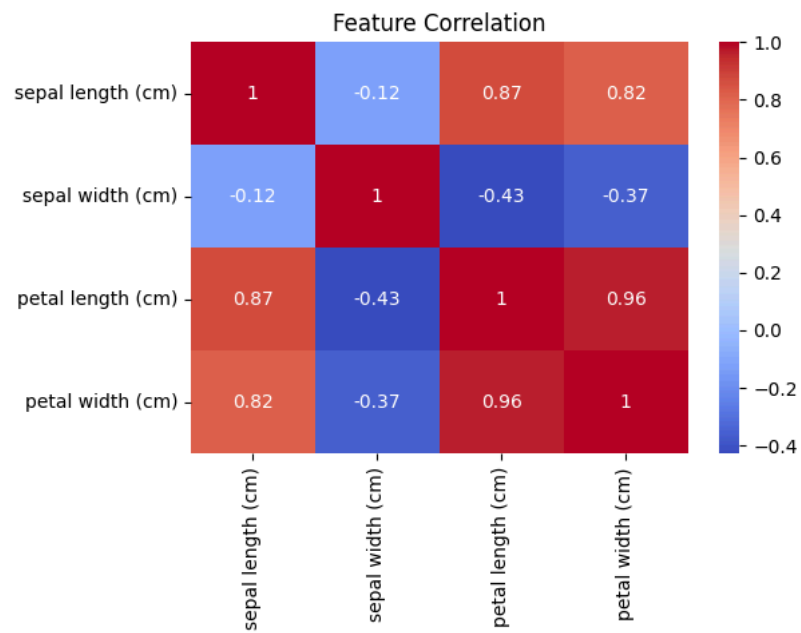
0 → setosa

1 → setosa

2 → setosa

3 → setosa

4 → setosa



Accuracy: 1.0

precision recall f1-score support

setosa	1.00	1.00	1.00	10
--------	------	------	------	----

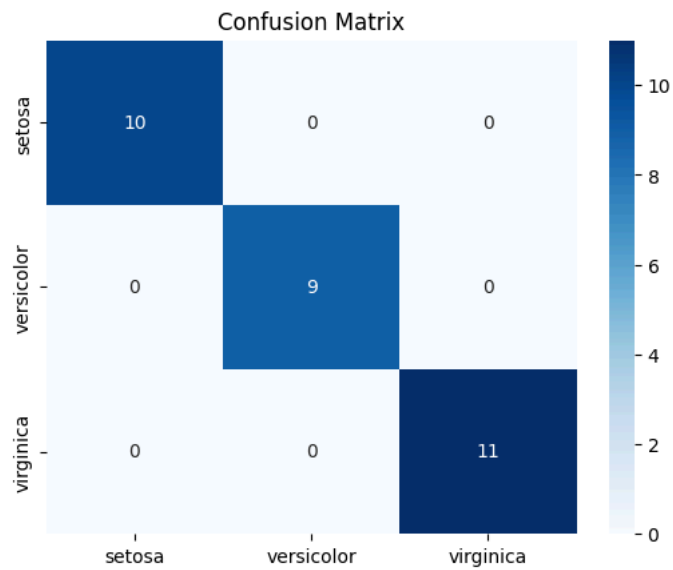
versicolor 1.00 1.00 1.00 9

virginica 1.00 1.00 1.00 11

accuracy			1.00	30
----------	--	--	------	----

macro avg 1.00 1.00 1.00 30

weighted avg 1.00 1.00 1.00 30



2. Handwritten digit recognition with MNIST dataset

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

digits = load_digits()
X,y = digits.data,digits.target
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

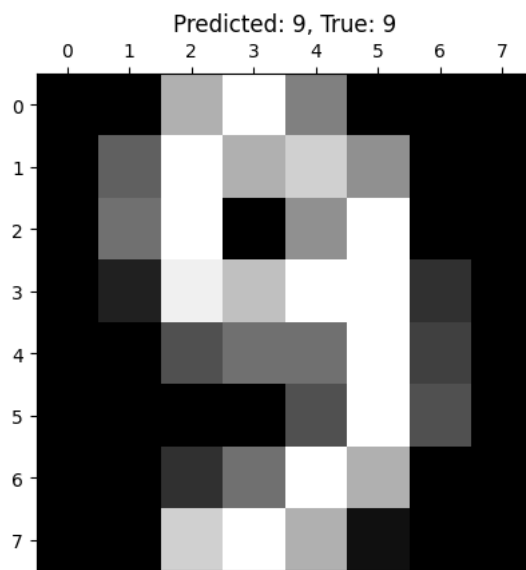
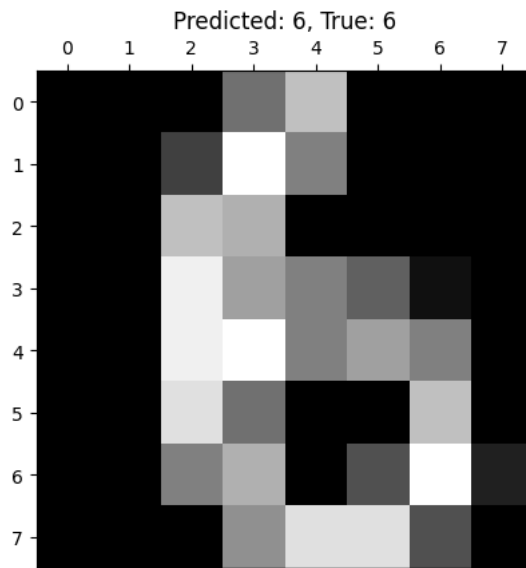
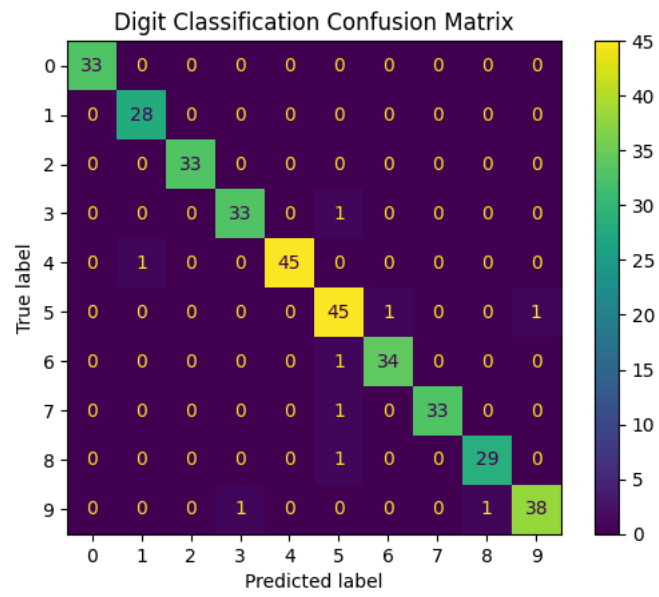
print("Accuracy:", accuracy_score(y_test,y_pred))

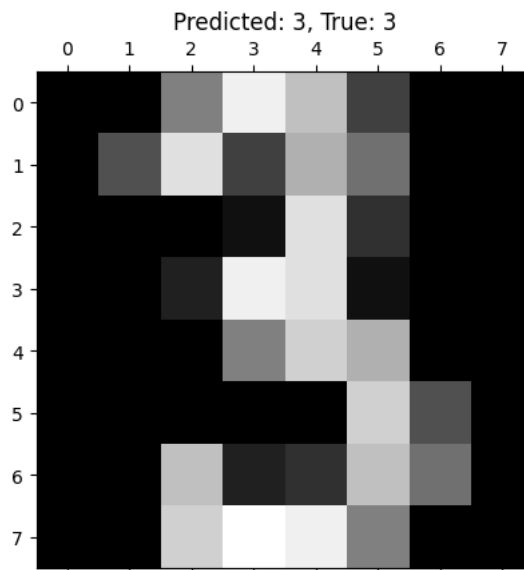
# Confusion matrix display
ConfusionMatrixDisplay.from_predictions(y_test,y_pred)
plt.title("Digit Classification Confusion Matrix")
plt.show()

# Sample predictions
plt.gray()
for i in range(3):
    plt.matshow(X_test[i].reshape(8,8))
    plt.title(f"Predicted: {y_pred[i]}, True: {y_test[i]}")
    plt.show()
```

Output:

Accuracy: 0.975





3. House price prediction with simple linear regression and multiple linear regression

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

housing = fetch_california_housing(as_frame=True)
df = housing.frame
print(df.head())

# Simple Linear Regression
X_simple = df[['MedInc']]
y = df['MedHouseVal']
X_train, X_test, y_train, y_test = train_test_split(X_simple, y, test_size=0.2, random_state=42)

simple_model = LinearRegression()
simple_model.fit(X_train, y_train)
y_pred_simple = simple_model.predict(X_test)

print("Simple LR R²:", r2_score(y_test, y_pred_simple))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_simple)))

plt.scatter(X_test, y_test, alpha=0.5, label="Actual")
plt.plot(X_test, y_pred_simple, color='red', label="Predicted")
plt.legend(); plt.title("Median Income vs House Value")
plt.show()

# Multiple Linear Regression
X_multi = df[['MedInc', 'HouseAge', 'AveRooms', 'AveOccup']]
X_train, X_test, y_train, y_test = train_test_split(X_multi, y, test_size=0.2, random_state=42)

multi_model = LinearRegression()
multi_model.fit(X_train, y_train)
y_pred_multi = multi_model.predict(X_test)
```

```
print("Multiple LR R²:", r2_score(y_test,y_pred_multi))
print("RMSE:", np.sqrt(mean_squared_error(y_test,y_pred_multi)))

plt.scatter(y_test,y_pred_multi,alpha=0.5,color='green')
plt.xlabel("Actual");plt.ylabel("Predicted");plt.title("Multiple Regression: Actual vs Predicted")
plt.show()
```

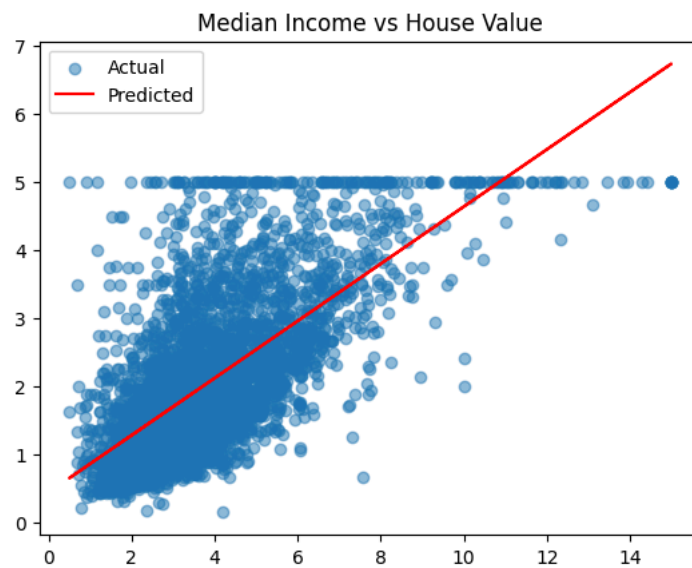
Output:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latit
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.8
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.8
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.8
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.8
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.8

	Longitude	MedHouseVal
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

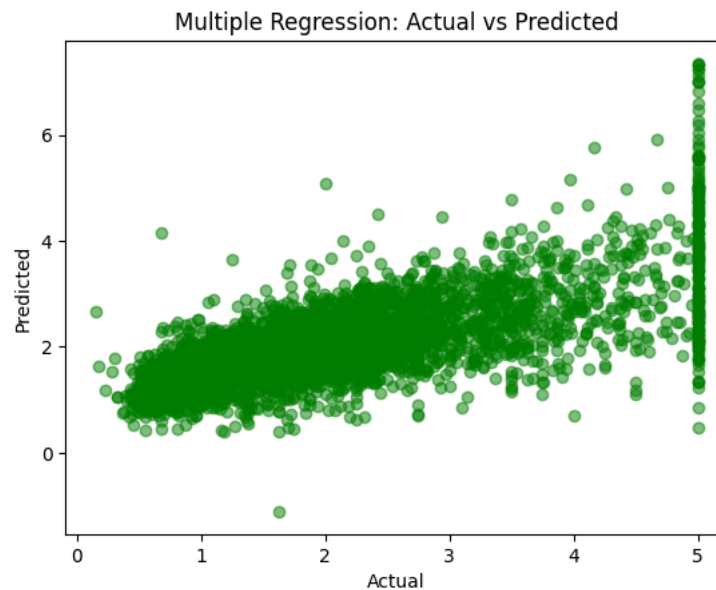
Simple LR R²: 0.45885918903846656

RMSE: 0.8420901241414455



Multiple LR R²: 0.49828508595474374

RMSE: 0.810833970602037



4. Stock price forecasting using historical market data

```
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import numpy as np

# === Historical stock price data for 15 days (same prices reused) ===
data = {
    'Day': list(range(1, 16)),
    'Price': [100,102,101,103,105,106,108,110,111,113,115,116,118,120,121]
}
df = pd.DataFrame(data)

# === Add a 3-day moving average as a feature ===
df['MA_3'] = df['Price'].rolling(3).mean().fillna(df['Price'])

# Features and target
X = df[['Day', 'MA_3']]
y = df['Price']

# === Train Linear Regression model ===
model = LinearRegression()
model.fit(X, y)

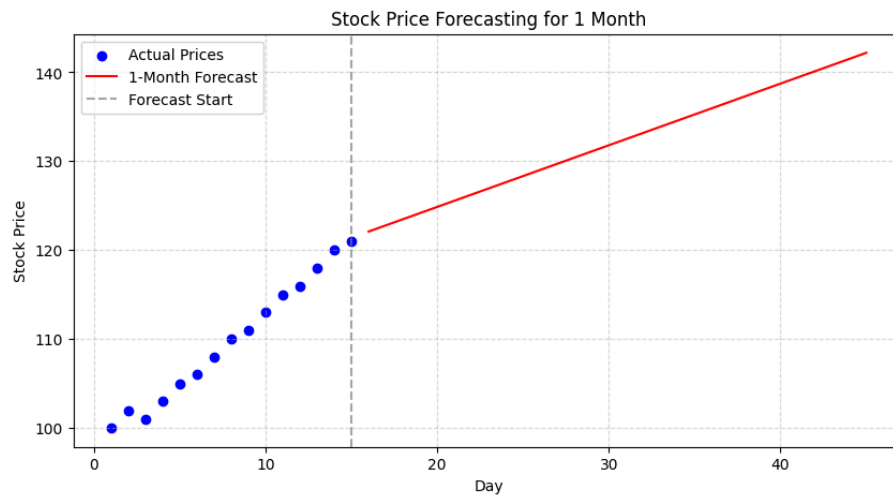
# === Forecast for next 30 days ===
future_days = np.arange(16, 46) # Days 16-45 (~1 month)
future_ma = [df['Price'][-3:].mean()] * len(future_days)
future_df = pd.DataFrame({'Day': future_days, 'MA_3': future_ma})

preds = model.predict(future_df)

# === Plot Actual vs Forecast ===
plt.figure(figsize=(10,5))
plt.scatter(df['Day'], df['Price'], color='blue', label='Actual Prices')
plt.plot(future_days, preds, 'r-', label='1-Month Forecast')
plt.axvline(x=15, color='gray', linestyle='--', alpha=0.7, label='Forecast Start')
```

```
plt.xlabel('Day')
plt.ylabel('Stock Price')
plt.title('Stock Price Forecasting for 1 Month')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

Output:



5. Predicting temperature, precipitation, and other weather metrics based on meteorological data

```
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

data = {'Humidity': [30, 40, 50, 60, 70],
        'WindSpeed': [5, 7, 6, 8, 7],
        'Temperature': [20, 22, 24, 26, 28],
        'Precipitation': [0, 1, 0, 2, 1]}
df = pd.DataFrame(data)

X = df[['Humidity', 'WindSpeed']]
y_temp = df['Temperature']
y_precip = df['Precipitation']

model_temp = LinearRegression().fit(X, y_temp)
model_precip = LinearRegression().fit(X, y_precip)

pred_temp = model_temp.predict(X)
pred_precip = model_precip.predict(X)

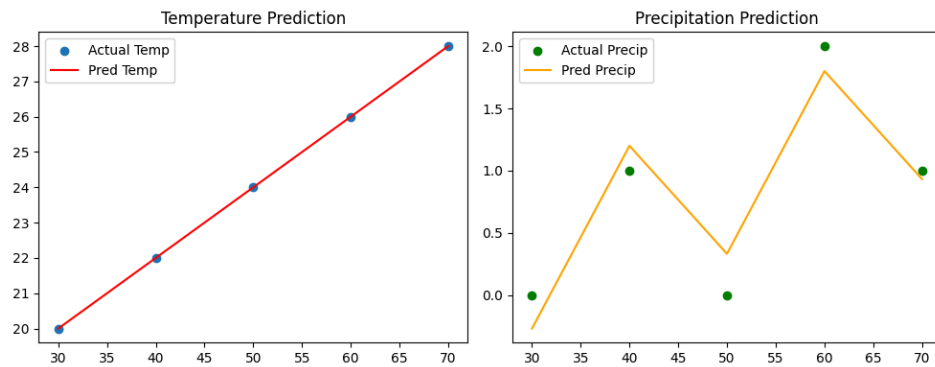
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.scatter(df['Humidity'], y_temp, label="Actual Temp")
plt.plot(df['Humidity'], pred_temp, 'r', label="Pred Temp")
plt.legend(); plt.title("Temperature Prediction")

plt.subplot(1, 2, 2)
plt.scatter(df['Humidity'], y_precip, label="Actual Precip", color="green")
plt.plot(df['Humidity'], pred_precip, 'orange', label="Pred Precip")
```



```
plt.legend();plt.title("Precipitation Prediction")
plt.tight_layout();plt.show()
```

Output:



6. Classifying patient data to predict the presence of a disease, such as a tumor being malignant or benign

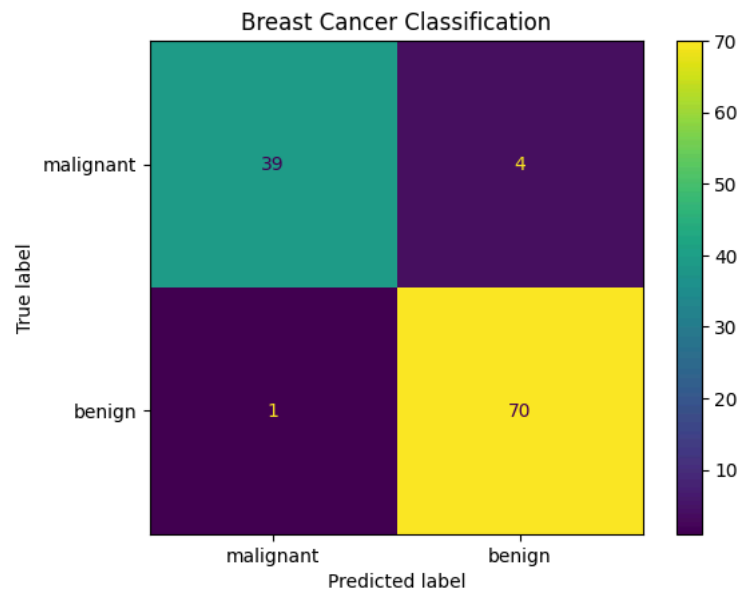
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

data = load_breast_cancer()
X,y = data.data,data.target

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
model = LogisticRegression(max_iter=5000)
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test,y_pred)*100:.2f}%")
ConfusionMatrixDisplay.from_predictions(y_test,y_pred,display_labels=data.target_names)
plt.title("Breast Cancer Classification")
plt.show()
```

Output:



Accuracy: 95.61%

7. Grouping customers with similar traits or purchasing behaviors to create target marketing strategies.

```
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import numpy as np

# === Expanded dataset: 25 customers with Age and Income ===
data = {
    'Age': [22,25,28,30,32,35,38,40,42,45,
            24,27,29,33,36,39,41,44,46,48,
            23,26,31,37,43],
    'Income': [20000,25000,30000,35000,40000,
               45000,50000,55000,60000,65000,
               22000,27000,32000,37000,42000,
               47000,52000,57000,62000,67000,
               24000,29000,34000,39000,44000]
}
df = pd.DataFrame(data)

# === Apply K-Means Clustering ===
# Increase clusters if needed, but here we keep 2 for simplicity
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(df[['Age','Income']])

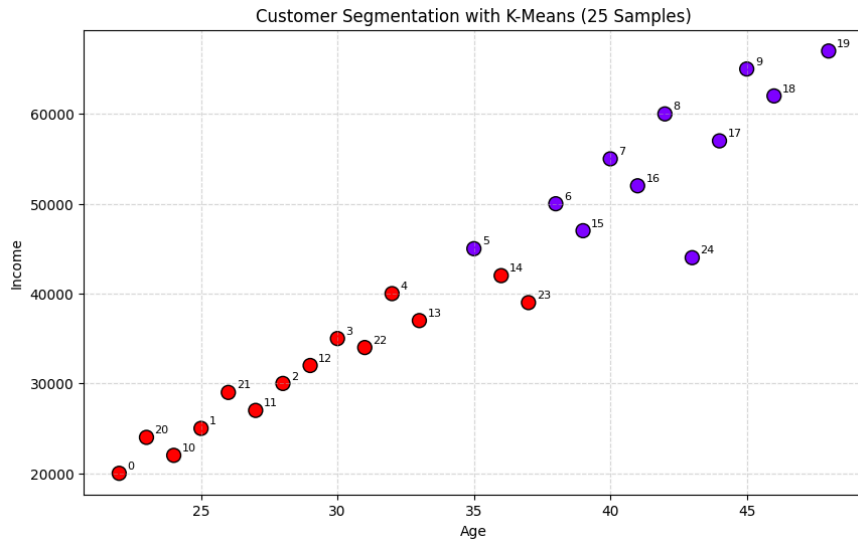
# Print cluster centers for inspection
print("Cluster Centers:\n", pd.DataFrame(kmeans.cluster_centers_, columns=['Age','Income']))

# === Plot the clusters ===
plt.figure(figsize=(10,6))
plt.scatter(df['Age'], df['Income'], c=df['Cluster'], cmap='rainbow', s=100, edgecolors='black')

# Annotate points for clarity
for i, row in df.iterrows():
    plt.text(row['Age']+0.3, row['Income']+500, str(i), fontsize=8)
```

```
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("Customer Segmentation with K-Means (25 Samples)")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

Output:



Cluster Centers

	Age	Income
0	41.909091	54909.090909
1	28.785714	31142.857143

- Learn a robot to navigate a maze or an environment with obstacles to reach a goal.

```
import numpy as np
import matplotlib.pyplot as plt
import random

# --- Maze Setup ---
# 0 = free space, -1 = obstacle, 10 = goal
maze = np.array([
    [ 0, 0, 0, 0],
    [ 0, -1, 0, -1],
    [ 0, 0, 0, 0],
    [-1, 0, -1, 10]
])

# State mapping: each (row,col) pair is a state
states = [(r,c) for r in range(4) for c in range(4)]
state_to_index = {s:i for i,s in enumerate(states)}
actions = ['up','down','left','right']

def valid_moves(state):
    """Return possible actions for a given state"""
    r,c = state
```

```

moves = []
if r>0: moves.append('up')
if r<3: moves.append('down')
if c>0: moves.append('left')
if c<3: moves.append('right')
return moves

def next_state(state,action):
    r,c = state
    if action=='up': r-=1
    elif action=='down': r+=1
    elif action=='left': c-=1
    elif action=='right': c+=1
    return (r,c)

def reward(state):
    r,c = state
    return maze[r,c]

# --- Q-Learning Parameters ---
Q = np.zeros((len(states), len(actions)))
alpha = 0.7    # learning rate
gamma = 0.9    # discount factor
epsilon = 0.2  # exploration rate
episodes = 1000

for ep in range(episodes):
    state = (0,0) # start always at top-left
    for step in range(50): # limit steps to avoid loops
        # Choose action: epsilon-greedy
        if random.random() < epsilon:
            action = random.choice(valid_moves(state))
        else:
            idx = state_to_index[state]
            # Only consider valid actions for argmax
            valid = valid_moves(state)
            valid_indices = [actions.index(a) for a in valid]
            qvals = [Q[idx,i] for i in valid_indices]
            action = valid[np.argmax(qvals)]

        next_s = next_state(state, action)
        r = reward(next_s)

        s_idx, a_idx = state_to_index[state], actions.index(action)
        ns_idx = state_to_index[next_s]
        Q[s_idx,a_idx] = Q[s_idx,a_idx] + alpha*(r + gamma*np.max(Q[ns_idx,:]) - Q[s_idx,a_idx])

        state = next_s
        if r == 10: # reached goal
            break

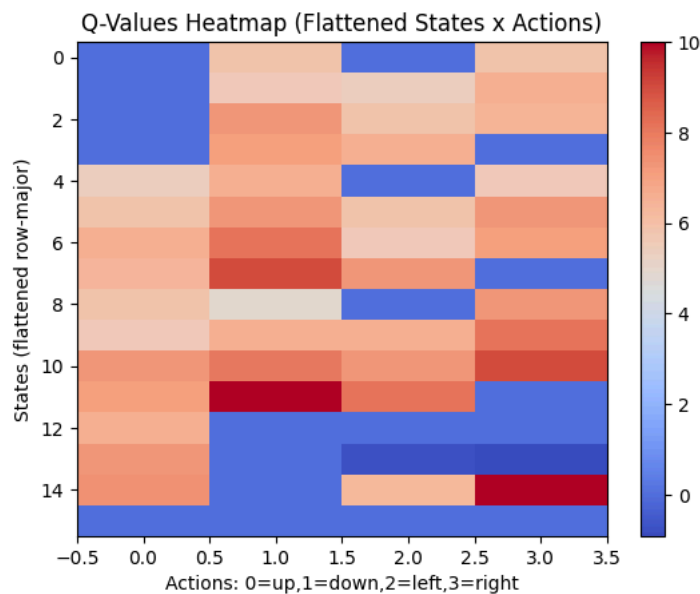
plt.imshow(Q, cmap='coolwarm', aspect='auto')
plt.colorbar()
plt.title("Q-Values Heatmap (Flattened States x Actions)")
plt.xlabel("Actions: 0=up,1=down,2=left,3=right")
plt.ylabel("States (flattened row-major)")
plt.show()

```

```
def run_episode():
    state = (0,0)
    path = [state]
    for _ in range(50):
        idx = state_to_index[state]
        a_idx = np.argmax(Q[idx,:])
        action = actions[a_idx]
        ns = next_state(state,action)
        path.append(ns)
        if reward(ns)==10:
            break
        state = ns
    return path

path = run_episode()
print("Learned path to goal:", path)
```

Output:



Learned path to goal: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (3, 3)]

9. Titanic survival prediction with data preprocessing

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Small Titanic-like dataset
data = {'Age': [22, 38, 26, 35, 28],
        'Sex': [0, 1, 1, 1, 0],
        'Survived': [0, 1, 1, 1, 0]}
df = pd.DataFrame(data)

X = df[['Age', 'Sex']]
y = df['Survived']
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42) # use larger test set

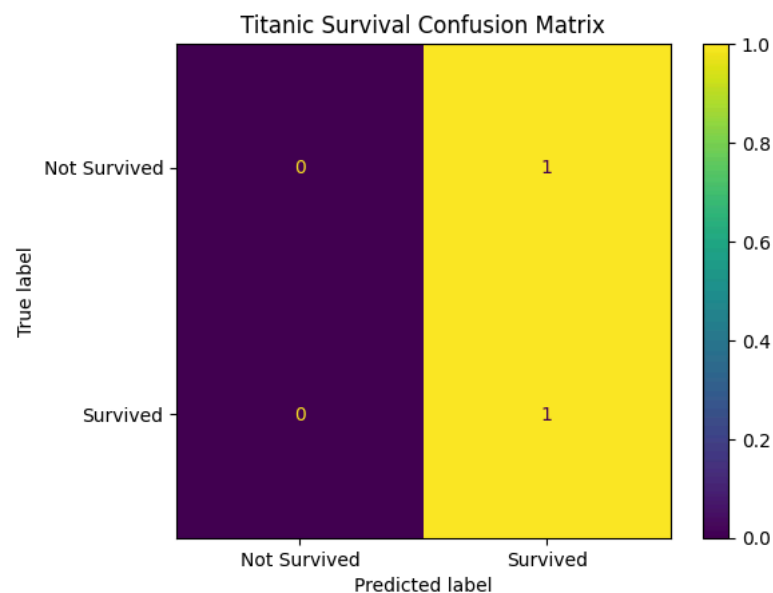
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%")

# ✅ Explicitly pass all labels to avoid warning
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred,
    labels=[0,1], # Ensure both classes are represented
    display_labels=['Not Survived','Survived']
)
plt.title("Titanic Survival Confusion Matrix")
plt.show()

```

Output:



Accuracy: 50.00%

10. Spam Detection

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

emails = ["Win money now","Cheap loan offer","Hi friend, how are you?",
          "Meeting tomorrow at 10am","Congratulations, you won!","Limited time offer, act now"]
labels = [1,1,0,0,1,1]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)
X_train,X_test,y_train,y_test = train_test_split(X,labels,test_size=0.33,random_state=42)

model = MultinomialNB()

```

```

model.fit(X_train,y_train)
y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test,y_pred)*100:.2f}%")
ConfusionMatrixDisplay.from_predictions(y_test,y_pred,labels=[0,1],display_labels=["Not Spam","Spam"])
plt.title("Spam Detection Confusion Matrix")
plt.show()

# Visualize word frequencies
word_counts = X.toarray().sum(axis=0)
words = vectorizer.get_feature_names_out()
plt.bar(words,word_counts)
plt.title("Word Frequency in Emails")
plt.xticks(rotation=45)
plt.show()

```

Output:

Accuracy: 100.00%

